

SafePay — Research Foundation

Payment Systems, Risk Landscape, SafePay's Positioning, and Banking Schema Patterns

Part 1: How Indian Payment Systems Actually Work

Why does India have four (really five, counting cheques) transfer systems?

It's not redundancy — it's segmentation by **urgency, value, and era of introduction**. Before RTGS existed (introduced 2004), banks used to individually transfer funds to each other, and this created huge systemic risk — if even one bank failed to pay, the domino effect would affect all banks. Each system since then was built to solve a specific gap the earlier ones left open:

NEFT (2005) solved the "no formal electronic system for retail transfers" gap. It started in November 2005, allowing retail bank customers to transfer funds between any two NEFT-enabled bank accounts, and even in 2017 was growing about 40% annually with total transaction value exceeding ₹120 lakh crore.

RTGS (2004) solved the "large payments need instant, individually-settled finality" gap — you don't want a ₹50 crore interbank payment sitting in a batch with retail transfers.

IMPS solved the "people want NEFT-like transfers but don't want to wait for the batch window" gap — it's a real-time instant inter-bank funds transfer system managed by the National Payments Corporation of India, available 24/7 throughout the year including bank holidays, unlike NEFT and RTGS.

UPI solved the "even IMPS is too clunky (account number + IFSC) for mobile-first daily payments" gap — it's a mobile-first instant payment system letting users send money via a simple Virtual Payment Address instead of full bank details, widely used for everyday transactions.

So the honest answer to "itne options diye hi kyu" is: each one is optimized for a different point on the urgency × value × convenience curve, and newer ones didn't retire older ones because their underlying settlement guarantees are genuinely different.

Core characteristics of each

NEFT

Works on a batch-processing mechanism — NEFT processes payments in half-hourly batches (up to 30 minutes).

Has no maximum limit for transfers, and is usually free for most bank transfers.

Best for: planned, non-urgent payments (salary, vendor payments, EMIs).

RTGS

Funds are settled instantly without waiting, unlike NEFT's batch processing.

Can be used for a minimum amount of ₹2 lakh with no upper limit — though RTGS removed its ₹2 lakh minimum in December 2020, so this varies by current policy.

Only available during banking hours historically, though most sources now note RTGS runs 24x7x365 following later RBI reforms.

Best for: high-value, time-sensitive payments where every minute counts.

IMPS

Can be used through internet banking, mobile banking, and ATMs, making it useful when UPI is unavailable, and is ideal for urgent transfers, especially when higher limits are required.

Has a ₹5 lakh per-transaction maximum limit (bank-dependent).

Best for: urgent, moderate-value transfers outside UPI's app ecosystem.

UPI

Supports up to ₹1 lakh for most users and suits daily mobile payments, and is free for everyday personal payments like transfers between individuals and regular bill payments.

Best for: everyday, small-ticket, high-frequency payments — QR codes, merchant payments, P2P.

Is UPI actually making payments "easy"?

Yes, on convenience — no full bank details needed, instant, free, works everywhere. But that same convenience is precisely what makes it the **highest-fraud-volume** rail in the country (covered in Part 2) — the friction UPI removed for legitimate users is the same friction that used to slow down scammers.

Part 2: Risk Landscape — What Actually Goes Wrong in Each Mode

NEFT / RTGS — the "wrong account number" risk

The single biggest risk here isn't hacking — it's **human error combined with a design choice**: banks and RBI have made it clear that even though the account holder's name is required along with the IFSC, NEFT/RTGS/IMPS transactions are solely based on the account number — if the account number entered is wrong, the amount goes to whatever account that number belongs to, and the transaction only fails if no such account exists. This is a structural gap RBI has since tried to patch: RBI has introduced a beneficiary account name look-up facility for RTGS/NEFT to reduce wrong credits and fraud, letting remitters verify the beneficiary's name using account number and IFSC before sending.

Once money lands in the wrong account, recovery is genuinely hard: once an RTGS transaction is settled it's treated as final, and any recovery must be pursued through a bank-led request to the beneficiary bank — not a cancellation. Even for NEFT, your bank asks the beneficiary's bank to reverse the credit, but by law that bank cannot debit the wrong receiver without their consent — so success depends entirely on the receiver agreeing to give the money back.

Other failure risks: incorrect beneficiary details causing rejection, insufficient funds, bank server/technical downtime, requests made outside the processing window, and transfers to frozen or dormant accounts.

Bank-side pain: every wrong-credit case becomes a manual, cross-bank, goodwill-dependent recovery process — expensive in staff time and reputational risk, with no guaranteed resolution.

IMPS — speed cuts both ways

RTGS and IMPS are near real-time, so money reaches the wrong account within minutes and there is little

chance of catching it "in transit" — unlike NEFT, which settles in batches and occasionally has a short window before settlement. So IMPS inherits NEFT/RTGS's "wrong account = hard to recover" problem, but *without even NEFT's small batch-window safety net*.

UPI — the highest-volume, most socially-engineered risk

This is where the numbers get serious. UPI frauds have surged in India, with over 13.4 lakh cases reported in the 2023-24 financial year, leading to losses exceeding ₹1,087 crore. Separately, since FY23, Indians have reported 2.7 million UPI fraud cases, resulting in losses of ₹2,145 crore. And underreporting is severe: 51% of UPI fraud victims did not file any official complaint with police, their bank, the platform, or regulators, suggesting the true scale is likely much higher than official statistics show.

The mechanism is almost always **social engineering, not system hacking**: UPI fraud typically requires the victim to actively participate — approving a request, scanning a QR code, or sharing an OTP — which is exactly why it's so hard to reverse; scammers don't hack the UPI system, they hack the user. Common patterns include:

Customers unknowingly sharing UPI PINs or OTPs with scammers posing as trusted entities.

Fake "Collect Request" messages pressuring users into entering their PIN and approving payments they never intended to make.

Scammers posing as bank officials or customer support, convincing victims to install remote screen-mirroring apps that hand over full control of the phone.

Money mule networks that use stolen UPI details to funnel funds through intermediary accounts, deliberately complicating recovery.

Regulatory backstop: RBI's rules on unauthorised transactions tie refund eligibility to reporting speed — full refund if reported within 3 days, limited liability of ₹5,000–₹25,000 if reported within 4–7 days, and bank-policy-dependent (customer may bear the loss) after 7 days.

Bank-side pain: dispute-handling volume at this scale is a major operational cost center, plus regulatory pressure — RBI is building a Digital Payments Intelligence Platform for real-time fraud detection, overseen by a committee led by former NPCI MD and CEO AP Hota specifically because the current reactive, post-fraud dispute process isn't scaling.

Risk summary table

Mode	Main risk	Why hard to fix	Recovery odds
NEFT	Wrong account (fat-finger)	Account-number based routing, receiver consent needed	Moderate, slow (days)
RTGS	Wrong account, high value	Instant + final settlement, no in-transit window	Low without receiver cooperation
IMPS	Wrong account, instant	No batch delay at all, near-zero catch window	Low

Mode	Main risk	Why hard to fix	Recovery odds
UPI	Social-engineering fraud	Victim actively authorizes it — looks "legitimate" to the system	Time-boxed (RBI 3/7-day rule), otherwise poor

Part 3: How SafePay Changes This — and Its Honest Limitations

Where SafePay genuinely helps

Fat-finger/NEFT-RTGS-IMPS wrong-account errors — SafePay's protection window is a direct answer to exactly the problem described above: once authorized, give the customer a few seconds to catch "wait, that's the wrong account" *before* the money is irreversibly gone, instead of the current reality where recovery depends on a stranger's goodwill.

UPI social-engineering fraud — this is arguably the stronger use case. Scam scripts work by creating urgency ("act now or you'll lose your money"). A mandatory pause — especially one that explicitly tells the user *why* it's pausing ("new beneficiary, unusually high amount") — interrupts that urgency script and gives the rational brain a moment to catch up, which is precisely the psychological lever scammers exploit.

Reduced dispute-handling load on banks — every payment cancelled *during* the protection window is one that never becomes a formal dispute, recovery request, or ombudsman complaint. This is a real cost-saving argument you can make to a bank stakeholder.

Corporate maker-checker — catches internal errors and insider fraud before a large payment leaves the organization, independent of external fraud entirely.

Honest cons — put these explicitly in your "Future Scope" section

SafePay cannot delay a real UPI/IMPS/RTGS settlement today. These rails are built for near-instant finality by NPCI/RBI infrastructure your project has no authority over. What SafePay can do in a real deployment is sit at the *bank's own initiation layer* — i.e., the bank pauses *before* it calls the NPCI/RTGS API — not intercept money already in flight on the interbank rail. This is worth stating plainly rather than implying SafePay reverses live transactions.

Friction vs. genuine urgency trade-off. A real emergency payment (medical, time-critical business payment) forced through a 60-second hold could itself cause harm. The policy engine needs an override path, and that override path is itself a new attack surface (a scammer coaching the victim to just hit "Release Now" defeats the whole mechanism).

Risk engine calibration is hard in practice. False positives (annoying, low-risk payments delayed) erode user trust in the product; false negatives (genuinely risky payments waved through) defeat the purpose. This needs real transaction data to tune properly — a cold-start problem for any new user or new bank.

Scammer adaptation. Since UPI fraud is social-engineering-driven, a sufficiently coached victim can be told exactly how to click through your protection screen. SafePay raises the bar; it doesn't eliminate the underlying vulnerability (human trust manipulation).

Regulatory/integration reality. Actually deploying this inside a real bank requires NPCI/RBI cooperation, security certification (VAPT), and probably a formal RBI sandbox process — not something a standalone app can do unilaterally. Worth acknowledging this is a *proposal/prototype* aimed at demonstrating the concept, not a production-ready bank integration.

Doesn't address the receiving end. Even a perfect protection window on the sender's side doesn't stop money mule accounts from existing — that's a separate KYC/AML problem SafePay doesn't claim to solve.

Framing all six of these explicitly as **Future Scope / Limitations** in your report is exactly what will make the project read as mature and well-considered rather than overclaiming — evaluators notice self-awareness like this.

Part 4: Database Schema — How Real Payment Systems Store This Safely

This is the single most important section for making SafePay's data layer look production-grade rather than a toy CRUD app.

The core principle: separate the ledger from the account view

A pattern many cloud-native cores adopt is separating the General Ledger (GL) from the customer-facing account layer — the GL enforces double-entry at the system level, while the account layer aggregates and presents balances to customers and APIs. Practically: use DECIMAL/NUMERIC for every monetary column (never FLOAT), enforce foreign-key constraints at the database level, and treat the transaction table as append-only from day one.

The ledger is immutable — you never edit or delete a posted row

The ledger design guarantees that every movement of funds is recorded in a balanced, immutable, and traceable manner — by enforcing double-entry and keeping transaction records read-only post-entry, integrity is ensured. If a transaction is wrong, you **post a new reversal entry** — you don't touch the original.

Idempotency — the most important column pattern for your project

Each transaction is identified by a unique reference (transaction ID or GUID); the posting logic is idempotent, meaning if the same transaction message is received twice due to a network retry or error, the system recognizes the duplicate and does not double-post it, typically by checking for an existing transaction ID before creating a new entry. The recommended implementation: require an idempotency key from upstream systems, store it as an `external_id` on the journal entry, and put a unique database constraint on the combination of (`source_system`, `external_id`) — also persist enough response data so repeat calls can be answered without recomputing.

Recommended schema for SafePay's ledger layer

`ledger_entry` (append-only, immutable)

Column	Type	Notes
id	NUMBER (PK)	
transaction_id	VARCHAR2(64)	your SafePay transaction reference
external_reference	VARCHAR2(64)	UTR (NEFT/RTGS) or RRN (IMPS/UPI) — the real interbank reference
source_system	VARCHAR2(30)	e.g. 'SAFEPAY', 'NEFT', 'UPI'
idempotency_key	VARCHAR2(64)	UNIQUE with source_system — prevents double-posting on retry
account_id	NUMBER (FK)	
entry_type	VARCHAR2(10)	DEBIT / CREDIT
amount	NUMBER(18,2)	never FLOAT
currency	VARCHAR2(3)	e.g. 'INR'
status	VARCHAR2(20)	POSTED / REVERSED
reversal_of_entry_id	NUMBER (nullable, FK to self)	points to the original entry being reversed
created_at	TIMESTAMP	write-once
effective_at	TIMESTAMP	when the movement is considered to have happened

Constraint: `UNIQUE (source_system, idempotency_key)` — this single constraint is what prevents duplicate postings from network retries, which is a very real failure mode in payment systems.

Handling failed / bounced / incorrect transactions — the exception table

Real systems don't try to cram failure-handling into the same row as a successful transaction. They use a **separate exception/reconciliation table**:

`transaction_exception`

Column	Type	Notes
id	NUMBER (PK)	
transaction_id	VARCHAR2(64) (FK)	links back to the original attempted transaction
failure_stage	VARCHAR2(30)	e.g. AUTHORIZATION, RAIL_SUBMISSION, SETTLEMENT_CONFIRMATION
error_code	VARCHAR2(20)	e.g. INVALID_IFSC, ACCOUNT_FROZEN, TIMEOUT, INSUFFICIENT_FUNDS
error_message	VARCHAR2(255)	human-readable detail
debited_not_credited	CHAR(1)	Y/N — flags the "money left my account but didn't arrive" scenario
retry_count	NUMBER	
resolution_status	VARCHAR2(20)	PENDING / AUTO_REVERSED / MANUAL_REVIEW / RESOLVED
resolved_at	TIMESTAMP	
created_at	TIMESTAMP	

This maps directly onto real failure categories you already researched: incorrect beneficiary details, insufficient funds, bank server issues, exceeded transaction time window, and frozen/dormant accounts.

Reconciliation — comparing your internal truth to the external rail

Reconciliation compares your ledger to external truth, like bank statements and settlement files. For SafePay, this means a periodic job (or table) that matches your internal `ledger_entry` rows against the real NEFT/RTGS/UPI settlement file/UTR — flagging anything that doesn't match into `transaction_exception`.

How this maps onto your existing SafePay entities

Your `Transaction` entity from the implementation guide is essentially the **account-facing view** — good for UI and business logic. What this section adds is the **ledger layer underneath it**: every time a `Transaction` moves to `SETTLED`, you'd post a balanced pair of `ledger_entry` rows (debit sender, credit beneficiary), and every time something goes wrong, it lands in `transaction_exception` instead of silently disappearing. This two-layer design (mutable business-status table + immutable ledger + exception table) is what separates a student project from something that *looks* like it understands how real banks actually protect money.

Suggested structure for your final report

Payment Systems Landscape (Part 1) — establishes you understand the domain

Risk Analysis per Mode (Part 2) — this is your "problem statement" evidence, cite the fraud statistics

SafePay's Value Proposition + Honest Limitations (Part 3) — shows maturity, gives you a clean Future Scope section

Data Architecture (Part 4) — shows technical depth beyond a basic CRUD schema

If you want, I can also pull the actual RBI Master Direction PDF for direct citation, or turn Part 4 into updated JPA entity code (`ledger_entry` + `transaction_exception`) that plugs into the implementation guide you already have.